CAHIER DES CHARGES

RETAILPULSE

Conception et déploiement d’une plateforme distribuée
de traitement des données e-commerce en quasi-temps réel

Formation	Mastere 2 — Data Sup de Vinci
Module	Données distribuées
Nature du projet	Projet de synthèse et mise en situation professionnelle
Organisation	Travail en binôme
Modalités	Conception • développement • déploiement • expérimentation • soutenance

ALIMOU DIALLO

Données distribuées Mastere 2 DATA sup de vinci

1. Contexte et problématique

Les plateformes e-commerce produisent continuellement des données de référence et des événements métier : catalogue de produits, clients, commandes, paiements et annulations. Leur exploitation rapide est essentielle au pilotage commercial, mais soulève des enjeux de volume, de latence, de fiabilité, de cohérence et de résilience.

RetailPulse vise à concevoir une plateforme distribuée pédagogique qui collecte des données depuis une API externe, simule une activité commerciale, transporte les événements avec Apache Kafka, les traite avec Spark Structured Streaming et conserve les résultats dans PostgreSQL. Un tableau de bord permet d'observer l'activité et les agrégations calculées.

Problématique — Comment construire une chaîne de données distribuée, depuis une source externe jusqu'à des indicateurs consultables, en maîtrisant la qualité des données, les doublons, la cohérence, la persistance et la reprise après incident ?

2. Objectifs pédagogiques

- Intégrer une API REST et comprendre ses structures JSON, sa pagination et ses limites.
- Concevoir des événements métier identifiés et horodatés.
- Mettre en œuvre Kafka en mode KRaft, organiser les topics et justifier le partitionnement.
- Traiter les flux avec Spark Structured Streaming : validation, filtrage, enrichissement, déduplication et fenêtres temporelles.
- Persister les référentiels et les résultats analytiques dans PostgreSQL avec contraintes et upsert.
- Conteneuriser les composants et déployer une architecture fonctionnelle dans Kubernetes.
- Construire un tableau de bord et interpréter les indicateurs.
- Évaluer le débit, la latence, la résilience et les limites de la solution.

3. Périmètre du projet

Inclus. Collecte des produits et clients DummyJSON ; constitution des référentiels ; simulation de commandes et de paiements ; publication Kafka ; traitement Spark ; agrégations par catégorie sur fenêtres de cinq minutes ; stockage PostgreSQL ; visualisation ; Docker et Kubernetes ; tests et mesures.

Hors périmètre. Paiement réel, gestion d'un stock physique, persistance des écritures sur DummyJSON, cloud public obligatoire et architecture microservices métier exhaustive. Les commandes et paiements sont simulés.

Principe fondamental — DummyJSON fournit les données de référence. Les commandes et les résultats de paiement sont générés localement par le producteur ; ils ne sont pas créés par l'API.

4. Sources de données — API DummyJSON

L'API DummyJSON fournit des données fictives mais structurées, adaptées à un projet pédagogique. Les groupes projet doivent consulter la documentation officielle et exploiter les endpoints suivants.

Ressource	Endpoint	Données à exploiter	Utilisation
Produits	https://dummyjson.com/products?limit=0	id, title, description, price, discountPercentage, rating, stock, brand, category, thumbnail	Catalogue, enrichissement et analyses par catégorie/produit
Utilisateurs / clients	https://dummyjson.com/users?limit=0	id, firstName, lastName, email, age, address…	Référentiel client et association aux commandes simulées
Paniers — facultatif	https://dummyjson.com/carts?limit=0	id, userId, products, quantities, totals	Source complémentaire pour calibrer les scénarios

Documentation officielle : https://dummyjson.com/docs

- Récupérer l'ensemble des éléments disponibles ; examiner les champs total, skip, limit et le tableau de résultats.
- Gérer les erreurs HTTP, les délais d'attente et les réponses incomplètes.
- Normaliser les champs utiles, conserver les identifiants source et tracer les volumes collectés.
- Documenter les transformations et les limites de l'API.

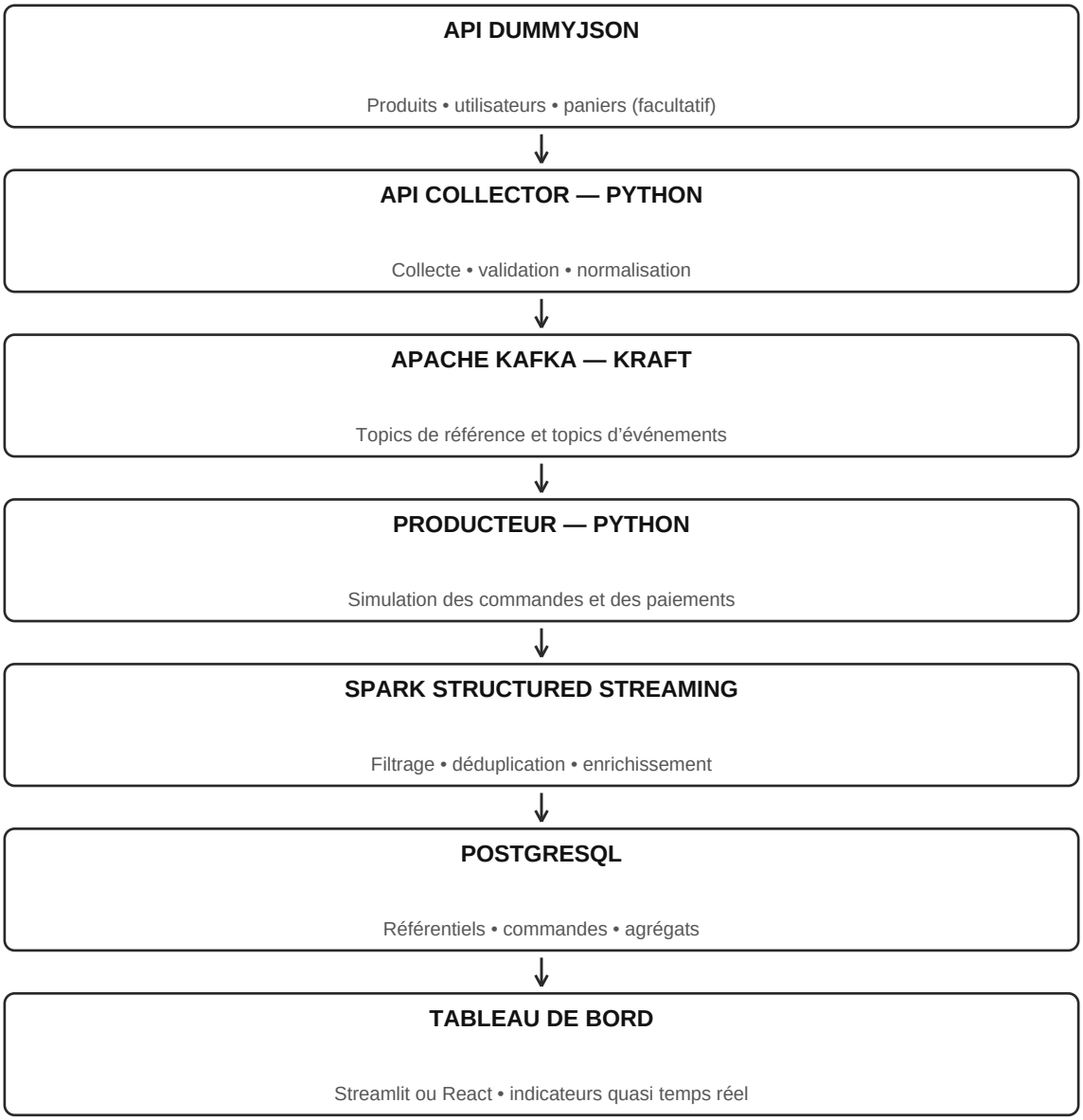
Attention. DummyJSON est une API de démonstration. Ses opérations d'écriture ne constituent pas une persistance métier durable. Les commandes et paiements sont simulés par le projet.

5. Architecture fonctionnelle de référence

Le schéma ci-dessous décrit le parcours logique attendu. Les groupes projet peuvent proposer des adaptations, à condition de préserver les fonctions essentielles et de justifier leurs choix.

SCHÉMA 1 — ARCHITECTURE FONCTIONNELLE

Chaîne de traitement des données, de la collecte jusqu'à la visualisation.



6. Composants et responsabilités

6.1 Collecteur Python — API Collector

- Interroger les endpoints produits et utilisateurs ; les paniers sont facultatifs.
- Valider et normaliser les attributs nécessaires au modèle relationnel.
- Publier les produits dans retail-products et les clients dans retail-customers, ou documenter un mécanisme de chargement initial équivalent.
- Tracer les volumes, les erreurs et la date de collecte ; fermer proprement les ressources Kafka.

6.2 Chargement des référentiels

- Rendre les produits et les clients disponibles pour enrichir les événements de commande.
- Un composant Reference Loader peut lire les topics de référence et insérer ou mettre à jour les données dans PostgreSQL.
- Le chargement doit être rejouable sans doublons, à partir des identifiants DummyJSON et d'un mécanisme d'upsert.

6.3 Producteur Python — simulation commerciale

Le producteur exploite les référentiels, sélectionne des clients et des produits, puis génère une activité commerciale continue. Il doit pouvoir être arrêté proprement et ses paramètres de simulation doivent être documentés.

- Se connecter à PostgreSQL pour charger les produits et les clients, ou utiliser les référentiels disponibles selon l'architecture retenue.
- Générer une commande avec order_id, client, un ou plusieurs produits, quantités, prix et horodatage.
- Publier ORDER_CREATED dans retail-orders.
- Publier le résultat du paiement dans retail-order-events : PAYMENT_COMPLETED, PAYMENT_FAILED et éventuellement ORDER_CANCELLED.
- Attendre un intervalle paramétrable (par exemple 0,5 à 2 secondes), puis répéter jusqu'au signal d'arrêt.

6.4 Kafka — transport des événements

Kafka doit être configuré en mode KRaft, sans ZooKeeper. Les groupes projet créent et testent les topics, choisissent le nombre de partitions et justifient la clé de partitionnement (par exemple order_id ou customer_id). Le facteur de réplication doit être adapté aux ressources disponibles et documenté.

7. Contrat des événements

Chaque événement métier est publié sous forme de message JSON. Le schéma exact peut être adapté, mais les champs d'identification, de temporalité et de traçabilité sont obligatoires.

Champ	Description et règle attendue
event_id	Identifiant unique de l'événement ; utilisé pour la déduplication.
event_type	Type métier : ORDER_CREATED, PAYMENT_COMPLETED, PAYMENT_FAILED, ORDER_CANCELLED…
event_timestamp	Date et heure de production, dans un format temporel documenté.
order_id	Identifiant de commande reliant les événements d'un même cycle.

Champ	Description et règle attendue
customer_id	Identifiant du client issu du référentiel.
product_id / items	Produit(s), quantité(s) et prix unitaire(s) associés à la commande.
category	Catégorie du produit, éventuellement enrichie depuis le référentiel products.
amount / currency	Montant et devise ; préciser si le montant est brut, remisé ou payé.

Règle de chiffre d'affaires — Seuls les événements PAYMENT_COMPLETED sont comptabilisés. PAYMENT_FAILED et ORDER_CANCELLED ne doivent pas augmenter le chiffre d'affaires. Si un paiement complété peut ensuite être annulé, la règle de compensation doit être définie.

Déduplication. Un event_id rejoué ne doit pas être comptabilisé plusieurs fois. Le groupe précise où la déduplication est appliquée et démontre son efficacité par un test.

8. Traitement Spark Structured Streaming

- Consommer les topics nécessaires, notamment retail-orders et retail-order-events.
- Parser le JSON et contrôler les champs obligatoires ; tracer ou isoler les messages invalides.
- Filtrer les événements de paiement pour ne retenir que PAYMENT_COMPLETED dans le calcul du chiffre d'affaires.
- Dédupliquer par event_id ; préciser la durée de conservation et les hypothèses associées.
- Enrichir les événements avec le catalogue produits pour disposer de la catégorie et des attributs utiles.
- Calculer des agrégations par catégorie sur des fenêtres de cinq minutes ; documenter le watermark et le traitement des événements tardifs.
- Écrire les agrégats dans PostgreSQL avec un mécanisme d'upsert.
- Configurer un checkpoint persistant et démontrer la reprise après redémarrage.

9. Modèle de données PostgreSQL

Le modèle relationnel minimal comprend quatre tables. Il s'agit d'une proposition de structuration cohérente avec les responsabilités du projet ; les groupes projet peuvent la faire évoluer et doivent justifier leurs choix.

Table	Rôle	Champs indicatifs	Contraintes / remarques
products	Catalogue	product_id (PK), title, category, price, stock, brand	Identifiant DummyJSON unique ; enrichissement des événements.
customers	Clients	customer_id (PK), first_name, last_name, email, city	Identifiant source unique ; email éventuellement unique selon les règles retenues.
orders	Commandes simulées	order_id (PK), customer_id (FK), created_at, status, total_amount	Lien vers customers ; préciser quel composant alimente la table.

Table	Rôle	Champs indicatifs	Contraintes / remarques
sales_aggregates	Résultats analytiques	category, window_start, window_end, order_count, total_quantity, total_revenue	UNIQUE(category, window_start, window_end) ; upsert.

Rôle de orders. Cette table représente le détail des commandes simulées. Elle n'est pas automatiquement alimentée par le calcul des agrégats : le groupe doit identifier le composant qui l'alimente. Si le stockage détaillé est hors du périmètre retenu, le choix doit être annoncé et justifié.

Indexation. Prévoir des clés primaires sur toutes les tables ; un index sur orders.customer_id et orders.created_at ; un index sur products.category si les filtres le justifient ; un index sur sales_aggregates.window_start et/ou category selon les requêtes du tableau de bord. Une contrainte UNIQUE sur event_id est pertinente si les événements sont persistés dans une table dédiée. Chaque index doit correspondre à un besoin identifié.

Anti-doublons et upsert. Définir les clés stables des référentiels, garantir l'unicité de l'agrégat par catégorie et fenêtre, et expliquer les effets d'un rejeu Spark.

10. Tableau de bord — exigences fonctionnelles

Le tableau de bord est un livrable obligatoire. Le binôme choisit **Streamlit** ou **React**. L'interface lit les données persistées dans PostgreSQL, directement ou via une API REST développée par le groupe. Elle ne doit pas recalculer les agrégations métier.

- Chiffre d'affaires total issu des paiements réussis.
- Nombre total de commandes et, si disponible, nombre de commandes payées.
- Nombre total d'articles vendus ou quantités cumulées.
- Nombre de catégories représentées.
- Évolution du chiffre d'affaires dans le temps, par fenêtres de cinq minutes.
- Répartition du chiffre d'affaires par catégorie.
- Top 5 ou top 10 des catégories ou produits les plus performants.
- Table des dernières agrégations ou commandes, avec horodatage et statut si disponible.
- Date/heure de dernière actualisation et état de disponibilité des données.

Actualisation. Fréquence raisonnable et documentée (par exemple 5 à 15 secondes). Afficher les unités et la devise ; gérer l'absence de données et les erreurs de connexion. Les indicateurs doivent respecter les règles métier du pipeline.

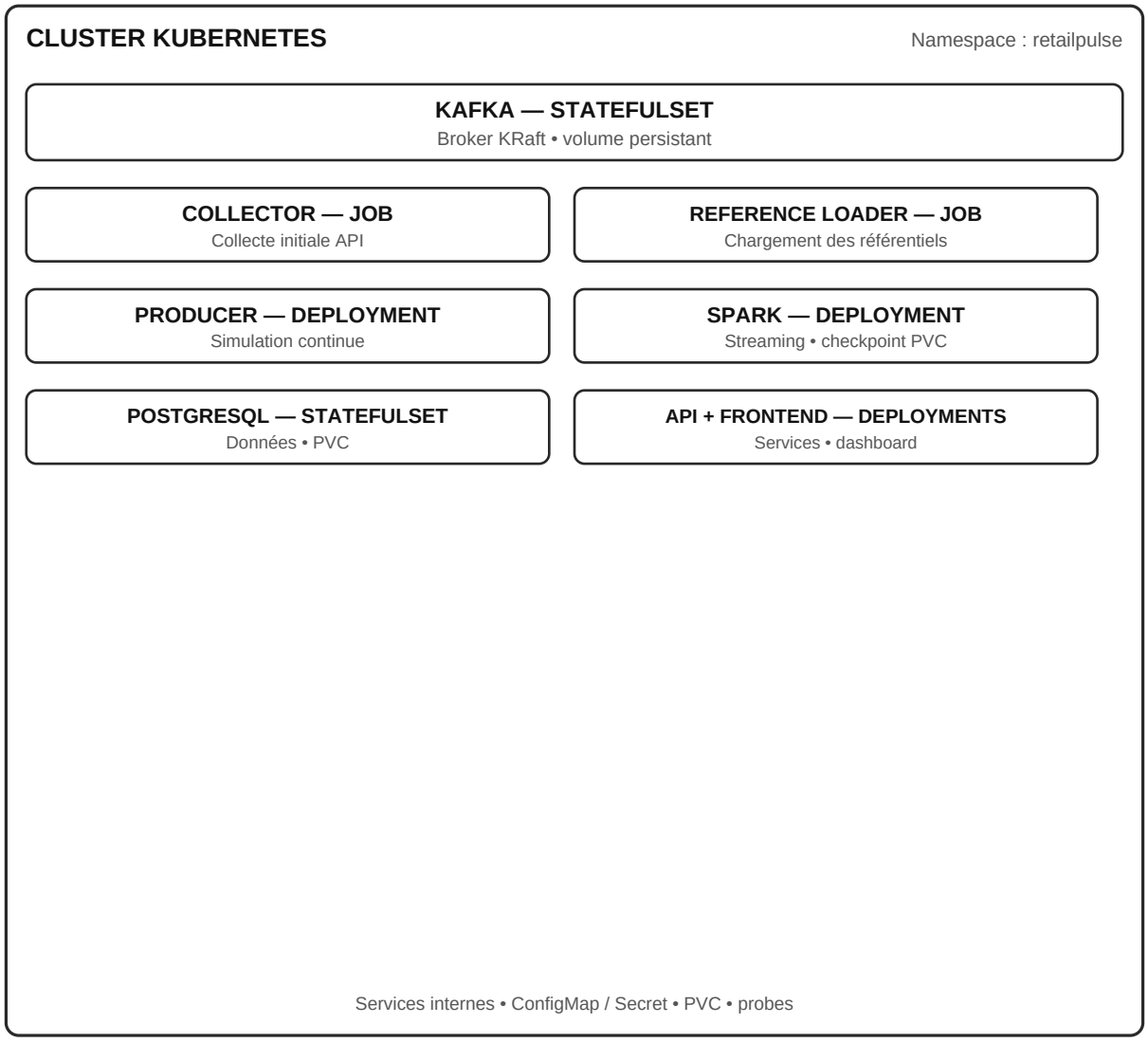
Critère d'acceptation — Une commande simulée doit traverser Kafka et Spark, être persistée dans PostgreSQL et devenir visible sur le tableau de bord sans intervention manuelle sur les données.

11. Conteneurisation et déploiement Kubernetes

Les composants applicatifs sont conteneurisés avec Docker. Kubernetes orchestre une partie représentative mais réellement fonctionnelle de la plateforme. Les ressources doivent être appliquées, observées et testées : des manifestes non déployés ne suffisent pas.

SCHÉMA 2 — ARCHITECTURE KUBERNETES

Organisation indicative des charges de travail et des services dans le cluster.



Ressource Kubernetes	Usage attendu	Points de vigilance
Deployment	Producer, Spark, API et frontend	Replicas, labels/selectors, images, ports, variables d'environnement, ressources CPU/RAM.
StatefulSet	Kafka KRaft et PostgreSQL si hébergés dans le cluster	Identité stable, volumes persistants et configuration.
Job	Collecteur initial et Reference Loader	Exécution ponctuelle, statut et relance maîtrisée.
Service	Communication entre composants	ClusterIP interne ; NodePort ou Ingress pour l'accès externe.
ConfigMap / Secret	Configuration / paramètres sensibles	Ne pas versionner de secrets en clair.
PVC	PostgreSQL, Kafka et checkpoint Spark	Vérifier montage, persistance et reprise.
Probes	Readiness et liveness	Ne déclarer prêt qu'un service réellement disponible.

Attendus de déploiement

- Construire et versionner les images des composants.
- Déployer dans un namespace dédié et organiser les manifestes par fonction.
- Définir les Services et utiliser les noms DNS Kubernetes pour les connexions internes.
- Prévoir des volumes persistants pour les données et checkpoints lorsque les composants sont dans le cluster.
- Définir des requests/limits compatibles avec les ressources disponibles.
- Exposer le tableau de bord via NodePort ou Ingress et documenter son accès.
- Vérifier Pods, Deployments, StatefulSets, Jobs, Services, PVC et logs.

Souplesse d'architecture. Les binômes peuvent déployer l'ensemble de la plateforme dans Kubernetes ou conserver certains services hors cluster. Dans ce cas, la frontière réseau et les communications doivent être clairement présentées et testées.

12. Fiabilité, cohérence et transactions

Le groupe doit expliciter les garanties et limites de chaque brique. Kafka transporte et conserve les messages selon sa configuration ; Spark traite et reprend selon son checkpoint et sa logique ; PostgreSQL assure les contraintes transactionnelles et la persistance ; Kubernetes redémarre ou replanifie les charges selon les contrôleurs et ressources déclarés. Aucun mécanisme isolé ne garantit l'absence de doublons de bout en bout.

- Définir la gestion des erreurs de collecte et de publication.
- Tracer les événements invalides ou impossibles à enrichir.
- Distinguer event_id, order_id et identifiants de référentiel.
- Documenter les événements tardifs et le watermark.
- Prévoir un arrêt propre du producteur et une reprise contrôlée.
- Démontrer l'idempotence du stockage des agrégats.

13. Tests et expérimentation

Scénario	Manipulation / entrée	Résultat attendu
Flux nominal	Démarrer collecte, producteur et Spark	Les messages circulent et les agrégats apparaissent dans PostgreSQL.
Paiement échoué	Générer PAYMENT_FAILED	Le paiement n'augmente pas le chiffre d'affaires.
Doublon	Publier deux fois le même event_id	L'événement n'est comptabilisé qu'une fois.
Reprise Spark	Arrêter puis relancer Spark avec checkpoint	Reprise conforme à la configuration, sans double comptage non maîtrisé.
Pod indisponible	Supprimer un Pod applicatif	Kubernetes le recrée si un contrôleur le prévoit ; impact analysé.
PostgreSQL indisponible	Interrompre puis rétablir le service	Erreurs, reprise et intégrité des données analysées.
Événement tardif	Publier un événement ancien	Comportement lié au watermark observé et documenté.

14. Mesure des performances

Réaliser au minimum deux expériences en faisant varier le débit de production, le nombre de partitions Kafka et/ou les ressources CPU/RAM. Les mesures doivent être reproductibles et interprétées.

- Débit : nombre d'événements produits et traités par seconde.
- Latence : délai entre émission et visibilité dans l'agrégat ou le tableau de bord ; méthode de mesure explicitée.
- Ressources : consommation CPU et mémoire des principaux composants.
- Comparer les configurations, identifier les goulots d'étranglement et proposer des améliorations.

15. Livrables attendus

Livrable	Contenu attendu
Dépôt Git	Code organisé, README, installation, configuration et contributions.
Dossier d'architecture	Schémas fonctionnel et Kubernetes, flux, modèle de données et justifications.
Collecte / référentiels	Collecteur DummyJSON, normalisation, gestion d'erreurs et chargement produits/clients.
Kafka / producteur	Configuration KRaft, topics, contrat JSON, producteur et simulation.
Traitement Spark	Structured Streaming, filtrage, déduplication, enrichissement, fenêtres et checkpoint.
PostgreSQL	Scripts SQL, schéma, contraintes, index justifiés et upsert.
Tableau de bord	Application Streamlit ou React et indicateurs demandés.
Docker / Kubernetes	Dockerfiles, manifestes, Services, ConfigMaps/Secrets, PVC et preuves de déploiement.

Livrable	Contenu attendu
Rapports	Tests, résilience, benchmarks, analyse et support de soutenance.

16. Démarche de réalisation recommandée

01	**Cadrage et conception** Analyser le besoin, définir les cas d'usage, données, événements et indicateurs ; produire les schémas et le modèle de données.
02	**Exploration de DummyJSON** Tester les endpoints, identifier les champs utiles, comprendre les réponses et formaliser la normalisation.
03	**Référentiels** Développer le collecteur et rendre produits/clients disponibles dans PostgreSQL et/ou Kafka ; vérifier le rejeu.
04	**Kafka KRaft** Configurer le broker, créer les topics, définir partitions et clés, valider publication et consommation.
05	**Simulation commerciale** Développer le producteur : commandes, paiements, identifiants uniques et horodatages.
06	**Spark Structured Streaming** Consommer, parser, valider, filtrer PAYMENT_COMPLETED, dédupliquer, enrichir et agréger par fenêtres de cinq minutes.
07	**Persistance PostgreSQL** Créer schéma, contraintes, index utiles et mécanisme d'upsert des agrégats.
08	**Tableau de bord** Construire l'interface Streamlit ou React et connecter les données persistées.
09	**Docker et Kubernetes** Construire les images, déployer les ressources, configurer Services, volumes, probes et exposition.
10	**Validation et soutenance** Exécuter les tests de résilience et de performance, analyser les résultats et démontrer le parcours complet.

17. Évaluation indicative

Critère	Pondération indicative
Conception, architecture et modèle de données	15 %
Collecte API et référentiels	10 %
Kafka, producteur et cohérence des événements	15 %
Spark, déduplication et agrégations	20 %
PostgreSQL, persistance et upsert	10 %

Critère	Pondération indicative
Docker, Kubernetes et déploiement	15 %
Tableau de bord	5 %
Tests, analyse et soutenance	10 %

La pondération peut être adaptée par le responsable pédagogique. La qualité de l’analyse, la justification des choix et la démonstration effective priment sur la quantité de fonctionnalités.

18. Soutenance et critères de réussite

Soutenance recommandée : 15 minutes maximum par binôme, suivies de questions. Chaque membre intervient et participe à la démonstration.

- Présenter la problématique et l’architecture.
- Montrer la collecte DummyJSON et les référentiels.
- Déclencher une commande et suivre ses événements dans Kafka.
- Expliquer le filtrage PAYMENT_COMPLETED, la déduplication et les agrégations Spark.
- Vérifier les données PostgreSQL et la mise à jour du tableau de bord.
- Présenter les ressources Kubernetes réellement déployées.
- Commenter un test de résilience et les mesures de performance.

Critère global de réussite — Le parcours de bout en bout est démontré ; les paiements échoués ne sont pas comptabilisés ; les doublons sont maîtrisés ; les agrégats sont persistés ; le tableau de bord est consultable à partir des données réellement produites.

19. Consignes générales

- Travail en binôme ; contributions identifiables dans Git.
- Documenter le rôle, les entrées, les sorties, les paramètres et les dépendances de chaque composant.
- Justifier les choix techniques et exposer les limites connues.
- Distinguer les données de référence DummyJSON des événements simulés.
- Ne pas déposer de mots de passe, clés ou secrets dans Git.
- Fournir des tests reproductibles et une documentation d’installation.
- Les extensions ne doivent pas compromettre les exigences obligatoires.

ALIMOU DIALLO
MASTER 2 — MODULE DONNÉES DISTRIBUÉES | PROJET RETAILPULSE